



sAlfer Lab

Joint lab on Safety and Security of AI

Py-01 · Introduzione a Python

Basi di Informatica, Gestione Dati e Statistica



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Argomenti

- Cos'è Python
- Variabili
- Input e output
- Tipi di dato
- Oggetti, metodi
- Istruzioni
- Esempi ed esercizi



sAlfer Lab

Joint lab on Safety and Security of AI

Cos'è Python

Linguaggio, codice sorgente, programma, interprete, compilatore



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Concetti di partenza

- Linguaggio di programmazione
- Codice sorgente
- Programma
- Interprete
- Compilatore
- Python

Linguaggio, codice sorgente, programma

Linguaggio di programmazione

Linguaggio formale usato per scrivere istruzioni che un computer può eseguire. Rappresenta in forma strutturata processi computazionali e permette di tradurre il codice in operazioni comprensibili ed eseguibili da una macchina.

Codice sorgente

Testo di un programma scritto in un linguaggio di programmazione leggibile dagli esseri umani (come Python, C o Java). Contiene le istruzioni che descrivono il comportamento del programma: è la forma originaria del software prima della sua esecuzione.

Programma (app)

Insieme organizzato di istruzioni scritte in un linguaggio di programmazione. Una volta eseguite da un computer, permettono di svolgere uno specifico compito o risolvere un determinato problema.

Interprete e compilatore

Interprete

Programma che esegue il codice sorgente traducendolo ed eseguendolo istruzione per istruzione durante il runtime.

Esempio: Python

Compilatore

Programma che traduce l'intero codice sorgente in anticipo in linguaggio macchina o in un codice intermedio, generando un file eseguibile o oggetto.

L'esecuzione avviene successivamente, senza bisogno del codice sorgente originale.

Esempio: C

Python

Linguaggio di programmazione di alto livello, interpretato, interattivo e orientato agli oggetti.

Con Python si realizzano piccoli programmi e script di base, ma anche programmi molto complessi. Esempi:

- Applicazioni desktop: applicazioni GUI e strumenti da riga di comando (CLI)
- Analisi matematiche e scientifiche dei dati, rappresentazione grafica dei risultati
- Applicazioni web
- Amministrazione di sistemi informatici e automazione di attività
- Attività DevOps

Perché Python

- Sintassi leggibile e adatta ai principianti
- “Batteries included”: la libreria standard include moduli per gestione dei file, networking, matematica, data e ora, test e molto altro
- Grande comunità e molte risorse: tutorial, video, forum e meetup locali per ogni argomento e livello di competenza
- Ampia diffusione nel mondo reale
- Versatile e scalabile
- Gratuito e multiplatforma
- Open source: Python Software Foundation License Version 2, con ampi diritti di utilizzo, modifica e distribuzione, anche in software proprietari. Chiunque può contribuire al suo sviluppo

Caratteristiche chiave

Interpretato

Il codice sorgente viene eseguito dall'interprete, istruzione per istruzione

Multiparadigma

Permette di scrivere codice in diversi stili: orientato agli oggetti, imperativo e funzionale

Tipizzazione dinamica

Controlla i tipi delle variabili a runtime: non è necessario dichiararli esplicitamente

Fortemente tipizzato

Non consente operazioni non sicure tra tipi incompatibili senza segnalarle

Installare Python

- Esistono diversi modi per installare Python e vari strumenti per scrivere il codice sorgente
- La scelta dipende da preferenze personali, sistema operativo e obiettivi di sviluppo

Thonny · thonny.org

Gratuito, multiplatforma e semplicissimo da installare

Non adatto a gestire progetti complessi, ma ottimo per le esigenze di un corso introduttivo

Online, senza installazione

Google Colab (colab.research.google.com): immediatamente pronto all'uso, è sufficiente un account Google

Altri servizi online (cercate "python online")

Ambienti di sviluppo (IDE) e distribuzioni

Alcuni esempi:

PyCharm

jetbrains.com/pycharm

Ambiente di sviluppo completo e professionale, adatto sia ai principianti sia a progetti complessi

Anaconda

anaconda.com

Distribuzione molto diffusa per l'analisi dei dati; include strumenti come Spyder e JupyterLab

Wing IDE

wingware.com

Ambiente di sviluppo leggero e professionale dedicato a Python

Per installazione e configurazione: documentazione ufficiale dei programmi scelti, per il proprio sistema operativo

Codice sorgente e commenti

Codice sorgente

Un semplice file di testo con estensione `.py`

Un programma Python può essere costituito da uno o più file di codice sorgente

Possono esserci altri file con informazioni aggiuntive: immagini, icone e altre risorse utili al programma

Commenti

Testo nel codice ignorato dall'interprete durante l'esecuzione

Descrive il funzionamento del codice, ne facilita la comprensione e chiarisce perché è scritto in un certo modo

```
# Questo è un commento!
```



sAlfer Lab

Joint lab on Safety and Security of AI

Variabili

Assegnazione, nomi, tipi numerici, booleani, stringhe

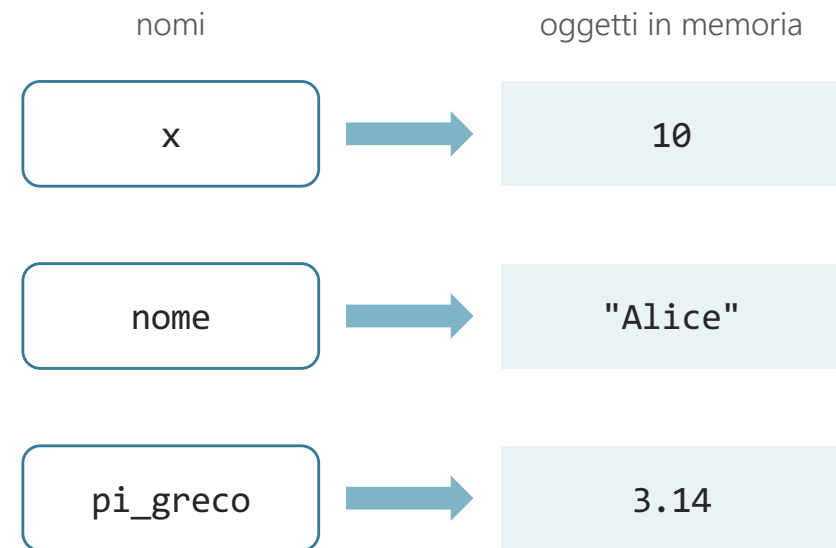


UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Variabili

- In Python le variabili sono nomi associati a un determinato oggetto
- Contengono un riferimento (o puntatore) all'indirizzo di memoria in cui l'oggetto è memorizzato
- Una volta assegnato un oggetto, vi si accede tramite il nome della variabile



Assegnazione

L'assegnazione è l'operazione che associa un valore a una variabile

Si effettua con l'operatore `=`, che non indica uguaglianza matematica: è un'associazione di riferimento tra un nome (la variabile) e un oggetto in memoria

```
x = 10  
nome = "Alice"  
pi_greco = 3.14
```

In questi esempi

`x` fa riferimento al valore intero 10

`nome` fa riferimento alla stringa "Alice"

`pi_greco` fa riferimento al numero 3.14

Cosa significa $x = x + 1$?

$$x = x + 1$$

Si calcola prima il lato destro (il valore attuale di x più 1), poi si associa il risultato al nome x

Non è un'equazione: è un aggiornamento

Nota importante: la variabile è un riferimento

In Python, diversamente da altri linguaggi di programmazione, la variabile non “contiene” direttamente il dato assegnato: memorizza un riferimento alla posizione in memoria in cui il dato è effettivamente memorizzato

La distinzione è sottile ma rilevante: la approfondiremo più avanti

Per semplicità si dice comunemente “la variabile contiene il valore x”, pur sapendo che tecnicamente l’affermazione non è corretta

Nomi delle variabili: regole

- Il nome dovrebbe indicare il significato del valore assegnato: `lista_utenti`, `numero_giorni`, `importo_totale`
- Qualsiasi lunghezza; ammessi lettere maiuscole e minuscole (`A-Z`, `a-z`), cifre (`0-9`) e underscore (`_`)
- Il primo carattere non può essere una cifra: deve essere un carattere alfabetico o l'underscore (per ora non useremo nomi che cominciano con `_`)
- Niente spazi né altri caratteri speciali come `!£$#%`

Nomi leciti e non leciti

Nomi leciti

```
a
a122
lista_utenti

un_elemento
_a1 # per ora non useremo variabili che cominciano con _
__a2 # per ora non useremo variabili che cominciano con __

unElemento #lecita ma non conforme a PEP8
ABC #lecita, ma secondo PEP8 il maiuscolo è riservato alle
COSTANTI
```

Nomi non leciti

```
3aa
valore!
somma totale
```

Variabili: convenzioni

- Oltre alle regole obbligatorie (es. un nome non può cominciare con un numero) esistono convenzioni che è bene adottare

snake_case

Nomi in minuscolo con parole separate da underscore

Es. `importo_totale`

MAIUSCOLO per le costanti

Nomi tutti maiuscoli per le costanti

Es. `MAX_VALORE`

PEP 8: peps.python.org/pep-0008/#function-and-variable-names · peps.python.org/pep-0008/

Tipi numerici e operatori aritmetici

- Python supporta numeri interi (`int`), numeri in virgola mobile (`float`) e numeri complessi (`complex`)
- Gli operatori rappresentano operazioni come addizione, sottrazione, moltiplicazione, divisione e così via

Operatori aritmetici

```
# le 4 operazioni: addizione,  
# sottrazione, moltiplicazione, divisione  
5 + 3  
5 - 3  
5 * 3  
5 / 3  
  
# Divisione con risultato intero  
# (Floor division)  
5 // 3  
  
# Modulo (rende il resto della divisione)  
5 % 3  
  
# Elevamento a potenza  
5 ** 3
```

docs.python.org/it/3/library/stdtypes.html#numeric-types-int-float-complex

Booleans

In Python i valori booleani sono solo due, **True** e **False**, implementati come sottoclasse degli interi: **True** si comporta come 1 e **False** come 0

```
print(True + True)    # 2
print(True == 1)      # True
```

Usiamo i booleani per esprimere il valore di verità di un'espressione

```
a = True
print(a)
a = False
print(a)
```

Assegnazione e confronto

L'operatore `=` (uguale) indica assegnazione; l'operatore `==` (doppio uguale) è una richiesta di confronto, il cui risultato può essere `True` o `False`

Un confronto produce come risultato un valore booleano, che possiamo stampare o memorizzare in una variabile

```
x = 10  
y = 20  
answer = x == y
```

L'operazione `x == y` produce il risultato `False`, che viene assegnato alla variabile `answer`

Operatori di confronto: esempi

```
a = 2 < 5
print(a)
a = 4 > 10
print(a)
a = 4 <= 3
print(a)
a = 3 >= 3
print(a)
```

```
a = 5 == 4+2 # attenzione a confronto vs
assegnazione
print(a)
a = 5 == 4+1
print(a)
a = 6 != 9
print(a)
a = (5 == 4) or (5 == 4+1)
print(a)
```

docs.python.org/it/3/library/stdtypes.html#boolean-type-bool

Stringhe

Le stringhe sono sequenze di caratteri definite usando virgolette (singole, doppie o triple)

L'operatore più (+) concatena le stringhe

```
a = 'hello'  
b = "world!"  
  
c = a+b  
print(c)
```

docs.python.org/it/3/library/stdtypes.html#text-sequence-type-str

Lunghezza di una stringa: len()

`len()` applicata a una stringa rende la lunghezza della stringa, cioè il numero di caratteri

Diversamente dal C, in Python non esiste un carattere nascosto di fine stringa: Python conta solo i caratteri che vediamo davvero. Gli spazi contano!

Per ora pensiamo a `len()` come a un comando che conta i caratteri di una stringa; più avanti vedremo cosa significa dire che `len()` è una "funzione"

```
s = 'hello world'
print(len(s))
```

Stringa in maiuscolo: upper()

```
s1 = 'Hello World!'
s2 = s1.upper()

print(s1)

print(s2)
```

Python fornisce anche altri tipi di dati: li vedremo in seguito

`upper()` permette di ottenere una nuova stringa in maiuscolo

Si scrive dopo la stringa, separato da un punto (dot notation): un comando usato così si chiama metodo

La stringa originale `s1` non viene modificata; `s2` contiene la versione maiuscola di `s1`

Solo le lettere minuscole vengono trasformate: spazi, numeri, simboli e lettere già maiuscole restano invariati



sAlfer Lab

Joint lab on Safety and Security of AI

Input e output

print(), input(), conversioni con int() e float(), primi programmi



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Input e output

Python mette a disposizione istruzioni per inviare informazioni in uscita (**output**) e acquisire informazioni dall'esterno (**input**)

I canali standard sono la tastiera e lo schermo, ma si possono usare tutte le periferiche I/O a disposizione

```
print('ciao, inserisci qualcosa da tastiera')  
# scrive sullo schermo  
  
a = input()  
# acquisisce da tastiera e inserisce il valore nella variabile  
  
print('Ciao, hai scritto', a)  
# scrive sullo schermo
```

Stringa o variabile?

```
print('Ciao, hai scritto', a)
```

'Ciao, hai scritto'

La sequenza di caratteri tra apici è una stringa

`print('Ciao, hai scritto')` visualizza
sullo schermo l'esatta sequenza di caratteri

`a`

`a` è una **variabile**, scritta senza gli apici:
`print()` mostra il valore memorizzato in `a`,
non la lettera a

Con `print('a')` il programma stamperebbe
il carattere a anziché il valore della variabile

Ancora su print()

Gli argomenti

Sono una sequenza di elementi separati da virgola

Ogni elemento è un valore o qualcosa che produce un valore: una stringa, un numero, una variabile, un'espressione (per ora, espressioni matematiche)

Come vengono stampati

Le stringhe così come sono

Le espressioni matematiche prima calcolate, poi stampate

Le variabili sostituite con il loro valore

Tra gli elementi, `print()` inserisce uno spazio; alla fine va a capo

`print()` offre molte altre opzioni per controllare l'output: formattazione del testo, dei numeri e modalità di stampa

print(): esempio

```
a = 10.1  
b = 2  
s = "Buon giorno!"  
  
print(s,"Il risultato è", a*b)
```

La stringa `s` è stampata così com'è, `"Il risultato è"` pure, `a*b` viene prima calcolato; tra gli elementi `print()` inserisce uno spazio

Riferimento ufficiale: Python Tutorial – Input and Output · docs.python.org/3/tutorial/inputoutput.html

Non tutte le operazioni sono possibili

```
print('ciao, inserisci un numero!')
a = input() # acquisisce da tastiera
b = a/2  # ERRORE! a è una stringa

print('hai inserito', a)
print('la metà è', b)
```

`input()` restituisce sempre una stringa, anche se l'utente digita un numero: occorre convertire la stringa in numero

È molto importante capire i messaggi d'errore dell'interprete: ci guidano verso la correzione dell'errore (anche all'esame!)

Altro esempio: cosa succede?

```
print('ciao, inserisci un numero!')  
a = input()  
  
print(a+a)
```

Le funzioni int() e float()

Convertono una stringa in intero o float (se la conversione è possibile)

float significa “virgola mobile”: il numero è rappresentato mediante mantissa ed esponente, come in $3.14 = 314 \cdot 10^{-2}$

È il modo in cui il calcolatore memorizza (e approssima) i numeri reali; nel calcolatore le potenze sono in base 2, non in base 10

I numeri reali possono essere rappresentati solo in maniera approssimata (perché?).

Conseguenza:

```
print(1-(1/3+1/3+1/3))
```

```
print(1-1/3-1/3-1/3)
```

Esempio di input con conversione

```
print('ciao, inserisci un numero!')

a = input() # acquisisce da tastiera
a = int(a) # oppure float(a) - converte in numero

b= a/2

print('hai inserito', a)
print('la metà è', b)

# alternativa: input e converto inline
# a = int(input())
```

Non sempre le conversioni sono possibili

```
a = "12"  
b = "13.55"  
c = "ciao"  
  
a1 = int(a) # 12 rappresentato come intero  
a2 = float(a) # 12.0 rappresentato come float  
  
b1 = int(b) # ERRORE. Tentativo di conversione a intero  
b2 = float(b) # 13.55 rappresentato come float  
  
c1 = int(c) # ERRORE. Tentativo di conversione a intero  
c2 = float(c) # ERRORE. Tentativo di conversione a float
```

Primi esempi di programma

```
# il mio primo programma
print("come ti chiami?")
nome_utente = input( )
stringa_finale='CIAO! '+nome_utente

print(stringa_finale)
```

```
# Secondo programma
print('dammi primo numero')
v1_s = input() # stringa
print('dammi secondo numero')
v2_s = input()

v1 = float(v1_s) # converto la stringa in float
v2 = float(v2_s)

risultato = v1 + v2

print('ecco la somma!')
print(risultato)
```



sAlfer Lab

Joint lab on Safety and Security of AI

Tipi di dato

type(), dove vengono memorizzati i dati, interning, keyword



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Tipi di dato

Python gestisce diversi tipi di dati (intero, float, stringa, ecc.): ogni tipo viene archiviato e manipolato in modo specifico

Python è **dinamicamente tipizzato**: non serve dichiarare esplicitamente il tipo di una variabile, Python lo deduce automaticamente dal valore assegnato

Il tipo di dato determina

come i dati vengono memorizzati in memoria

quali operazioni possono essere eseguite su di essi

come Python li gestisce internamente

La funzione type()

`type()` restituisce il tipo (la classe) dell'oggetto passato come argomento

```
a = 10
b = 11.11
c = 'hello'

print(a, ' tipo di dato: ', type(a))
print(b, ' tipo di dato: ', type(b))
print(c, ' tipo di dato: ', type(c))

# 10  tipo di dato:  <class 'int'>
# 11.11  tipo di dato:  <class 'float'>
# hello  tipo di dato:  <class 'str'>
```

I float sono un'approssimazione

Conseguenza pratica: i numeri reali possono avere infinite cifre decimali; la rappresentazione mediante **float** è necessariamente approssimata

```
a = 1-1/3-1/3 -1/3  
print(a) # teoricamente 0  
print(1/a) # teoricamente impossibile
```

Dove vengono memorizzati i dati?

```
a = 10
# memorizza 10 in memoria. `a` è un riferimento a quell'area di memoria.

b = a
# crea un nuovo riferimento alla stessa area di memoria.

print( hex(id(a)), hex(id(b)))
```

La funzione `id()` restituisce l'indirizzo di memoria a cui la variabile fa riferimento; `hex()` lo trasforma in base esadecimale (base 16)

Le due variabili fanno riferimento allo stesso indirizzo

In realtà `id()` restituisce un valore univoco per ogni oggetto. Nell'implementazione più diffusa (CPython) corrisponde, al momento in cui sono scritte queste note, all'indirizzo di memoria. L'importante è che `id()` fornisca un riferimento univoco per ogni oggetto

docs.python.org/it/3/library/functions.html#id

Numeri piccoli gestiti come singleton

Gli interi nel range da -5 a 256 sono trattati come **singleton** attraverso un meccanismo di caching (spesso chiamato informalmente **interning**)

Ogni volta che si assegna uno di questi valori a una variabile, Python riutilizza la stessa istanza in memoria: tutti i riferimenti a questi numeri puntano allo stesso oggetto

Attenzione: non è una caratteristica del linguaggio Python, ma un dettaglio implementativo dell'interprete. Con CPython, in genere gli interi in un certo intervallo sono memorizzati con l'**interning**; il comportamento non è garantito dal linguaggio e può variare tra implementazioni o versioni

docs.python.org/3/c-api/long.html#c.PyLong_FromLong

Interning: interi piccoli e float

Intero piccolo: Python riusa la stessa area di memoria

```
a = 10
# memorizza 10 in memoria.
# `a` è un riferimento a quell'area di memoria.

b = int("10")
# Python riusa la stessa area di memoria

print(hex(id(a)), hex(id(b)))
```

Float: il meccanismo di interning non viene utilizzato

```
a = 3.1
b = 3.1
# per i float non viene utilizzato
# il meccanismo di interning

print( hex(id(a)), hex(id(b)))
```

Interi grandi: nessun riuso della memoria

```
a = 123456789123456789
# memorizza il valore in memoria. `a` è un riferimento a quell'area di memoria.

b = a + 1
c = b - 1
# a e c hanno lo stesso valore,
# ma Python NON riusa la stessa area di memoria
# (non sono interi piccoli)
print(a == c)
print(hex(id(a)), hex(id(c)))
```

La variabile contiene un riferimento: Python e C

La variabile non contiene direttamente il valore, ma solo un riferimento: se cambio il valore, cambio il riferimento. Il meccanismo è diverso da quello del linguaggio C

Esempio in Python

```
a = 1
print( hex(id(a)))
# memorizza 1 in memoria. `a` è un
# riferimento a quell'area di memoria.

a = a + 1
print( hex(id(a)))
# memorizza 2 in una nuova area di memoria.
# `a` è un riferimento alla nuova area.
```

Esempio in C

```
int a;
a = 10;
printf("%p\n",&a ); //0x16b446f6c
a = a + 1;
printf("%p\n",&a ); //0x16b446f6c
```

Keyword

Le parole chiave (keyword) sono termini riservati dal linguaggio con un significato specifico: non possono essere usati come nomi di variabili, funzioni o identificatori

L'insieme è fissato per ciascuna versione del linguaggio

Non usatele come nomi di variabili nel codice!

```
>>> help("keywords")
```

```
Here is a list of the Python keywords.  
Enter any keyword to get more help.
```

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Errore subdolo

```
print = 10
```

Usa il nome della funzione `print` come nome di una variabile: impedisce l'uso della funzione `print()` fino al riavvio dell'interprete

Lo stesso vale per gli altri nomi predefiniti, come `input`, `len`, `type`

Le keyword vere invece non possono essere usate come nomi: `if = 10` produce un errore di sintassi



sAlfer Lab

Joint lab on Safety and Security of AI

Oggetti, metodi

Dato + comportamento · valore e identità: == e is · l'oggetto None



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Oggetti: dato + comportamento

In Python molti valori che assegniamo a una variabile sono **oggetti**: una combinazione di **dato** + **comportamento**

Un oggetto contiene un valore (per esempio una stringa o un numero) e mette a disposizione **operazioni applicabili a quel valore**: i **metodi**, richiamati con la **dot notation**

```
s.upper()
```

Con `s.upper()` chiediamo all'oggetto stringa `s` di restituire una versione tutta in maiuscolo

Un oggetto non è solo un valore memorizzato in una variabile, ma anche un insieme di funzionalità che sanno operare su quel valore

Ricorda: la variabile non contiene direttamente il dato (come in C), ma un riferimento all'area di memoria dell'oggetto. Diremo spesso "la variabile contiene l'oggetto": è un'approssimazione

Confronto tra oggetti: valore e identità

`==`

Verifica se due oggetti hanno lo stesso valore

Due stringhe con gli stessi caratteri sono uguali anche se sono oggetti distinti (in zone diverse della memoria)

`is`

Verifica se due variabili fanno riferimento allo stesso oggetto in memoria

Controlla l'identità dell'oggetto, non il suo contenuto

Per confrontare dei valori di solito usiamo `==`; `is` si usa più raramente, quando vogliamo verificare se due variabili puntano esattamente allo stesso oggetto

== e is: interi

Stesso oggetto

```
a = 10
b = a

print(a == b)    # True
print(a is b)    # True
```

Entrambi i confronti sono **True**: **a** e **b** si riferiscono allo stesso oggetto

Interi creati separatamente

```
a = int("30")
b = int("30")

print(a == b)    # True
print(a is b)    # probabilmente True
```

Con l'**interning**, i valori interi "piccoli" possono essere rappresentati dallo stesso oggetto in memoria: anche **is** rende **True**

`==` e `is`: interi grandi e float

Intero non piccolo

```
a = int("1000")
b = int("1000")

print(a == b)    # True
print(a is b)    # False
```

I valori sono uguali (`==`), ma si tratta di due oggetti distinti: `is` è `False`

Float

```
x = float("3.14")
y = float("3.14")

print(x == y)    # True
print(x is y)    # False
```

Valori uguali, oggetti diversi: per i `float` il meccanismo di `intern`ing non viene utilizzato

== e is: stringhe

Stringhe letterali

```
s1 = "ciao"
s2 = "ciao"

print(s1 == s2)    # True
print(s1 is s2)    # spesso True
```

Python può riutilizzare automaticamente le stringhe: a volte `is` restituisce `True`, ma il comportamento non è garantito

Stringa costruita a runtime

```
s1 = "ciao"
s2 = "ci"
s2 = s2+"ao"

print(s1 == s2)    # True
print(s1 is s2)    # False
```

Stesso contenuto, quindi `==` è `True`; ma `s2` è costruita a runtime per concatenazione: è un oggetto diverso da `s1`, quindi `is` è `False`

L'oggetto None

None è un oggetto speciale, usato spesso per indicare che

- una variabile non ha ancora un valore significativo
- una funzione non restituisce alcun risultato

None è un oggetto unico (singleton): nel programma esiste una sola istanza. Per verificare se una variabile vale **None** è buona pratica usare **is**

```
x = None

print (x is None)  # True

x= 1+2
print (x is None) # False
```



sAlfer Lab

Joint lab on Safety and Security of AI

Istruzioni

Istruzioni e flusso di esecuzione sequenziale



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Istruzioni

Un'istruzione (statement) è un'unità eseguibile di codice che Python può interpretare ed eseguire

Le istruzioni possono includere parole chiave, operatori e funzioni

Esempi semplici: l'assegnazione di un valore a una variabile, la stampa di un risultato sullo schermo

```
a = 10  
b = input()  
c = 2 * a  
print('Risultato', c)
```

Flusso di esecuzione sequenziale



```
a = 10  
b = input()  
c = 2 * a  
print('Risultato', c)
```

Python esegue le istruzioni in modo **sequenziale**: dall'alto verso il basso, nell'ordine esatto in cui sono scritte

L'ordine delle istruzioni è fondamentale: determina il comportamento del programma e i risultati prodotti

Più avanti vedremo come **alterare il flusso sequenziale** con condizioni, cicli e funzioni: per eseguire alcune istruzioni solo in determinate situazioni, ripeterle più volte o organizzarle in blocchi riutilizzabili



sAlfer Lab

Joint lab on Safety and Security of AI

Esempi ed esercizi

Otto programmi da scrivere con input, conversioni, operatori e print()



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

Esercizio 1: costo_colazione_1

Chiedere all'utente il numero di paste mangiate e il costo unitario; calcolare e stampare il costo finale. Stampare anche il tipo di dato e l'indirizzo di memoria

```
print('quante paste hai mangiato?')
stringa_numero_paste= input()
numero_paste = int(stringa_numero_paste)
print('costo unitario?')
stringa_costo_unitario= input()
costo_unitario = float(stringa_costo_unitario)

print('tipo della var stringa_numero_paste:', type(stringa_numero_paste))
print('tipo della var numero_paste:', type(numero_paste))
print('tipo della var costo_unitario:', type(costo_unitario))
print('locazione di memoria della var costo_unitario:', hex(id(costo_unitario)))

costo_totale = numero_paste * costo_unitario
print('costo totale', costo_totale)
```

Esercizi 2–4

2 · hello_1

Chiedere nome e cognome e salutare stampando 'Ciao' seguito da nome e cognome

Variante: stampare il numero di caratteri totali di nome e cognome. Suggerimento: `len(s)` rende la lunghezza di una stringa

3 · rettangolo_1

Chiedere la lunghezza dei lati di un rettangolo e stampare l'area e il perimetro

4 · triangolo_1

Chiedere base e altezza di un triangolo e stampare l'area

Esercizi 5–6

5 · triangolo_2

Chiedere l'area di un triangolo rettangolo isoscele e stampare il perimetro

Suggerimento: la radice quadrata si calcola così

```
import math
math.sqrt(x) # radice quadrata di x
```

6 · triangolo_3

Chiedere la lunghezza dei lati di un triangolo e stampare l'area e il perimetro

Esercizi 7–8

7 · eq_1

Chiedere i coefficienti di un'equazione di secondo grado nella forma $a \cdot x^2 + b \cdot x + c = 0$ e trovare le radici

Si assuma che l'equazione ammetta radici reali e distinte

8 · conversione_4bit_1

Chiedere il valore delle 4 cifre di un intero in base 2 e rendere il valore in base 10

Calcolare anche il prodotto per 2 e la divisione (intera) per 2, senza usare direttamente prodotto e divisione, ma la proprietà di shift a destra e sinistra